
Lab 9 – Polymorphism and Exception

Aim

This week is focused on polymorphism and throwing/handling exceptions. This builds on your object work from Lab 6 and Lab 8 – copy related classes such as shown in Lab 8 to start this lab from scratch.

Tips:

1. Complete the previous lab sheet before you start this sheet.
2. Ask a TA if you do not understand the lab tasks, or you cannot explain the output.

Lab Task 9.1 Polymorphic Function

Recall that last week, we have created a subclass of `StockPrice` named `StockPricePlus`, in which we have overridden the `toString()` method.

Consider the following function in `StockPricePlus` class:

```
public static void printStockPrice(StockPrice[] stockArray) {  
    for (int i = 0; i < stockArray.length; i++) {  
        System.out.println(stockArray[i]);  
    }  
}
```

The function takes an array of `StockPrice` objects, and simply calls `toString()` on each element of the array by printing everything out (recall that `toString()` will be automatically called when we `println` an object).

Now, in the main method, consider assigning some array elements to `StockPricePlus` instead. Recall that this is okay, we can use the superclass variable to refer to a subclass object.

```
public static void main(String[] args) {  
    StockPrice[] sPArr = new StockPrice[2];  
    sPArr[0] = new StockPrice("AAPL", "2007-12-1", 91.8, 92.33, 90.1, 91.32,  
                             28395700);  
    sPArr[1] = new StockPricePlus("AAPL", "2006-12-1", 91.8, 92.33, 90.1,  
                                 91.32, 28395700);  
    printStockPrice(sPArr);  
}
```

Question:

Which `toString()` methods will be called when we `println` a `StockPrice` variable that refer to a `StockPricePlus` object?

Answer:

Because of polymorphism, since `StockPricePlus` is a subclass of `StockPrice`, and we override `StockPrice`'s `toString()` in `StockPricePlus`, the overriding `toString()` of `StockPricePlus` will be called!

As you can see, this is very useful because a general function taking the superclass object can produce the subclass behavior using the same single method call – a very powerful code reuse feature of Object-Oriented Programming!

Lab Task 9.2 Throwing Exception

Consider the following program:

```
import java.util.Scanner;

public class ExceptionalLab {

    public static int readingInt() {

        Scanner sc = new Scanner(System.in);
        String input = sc.nextLine();
        int number = Integer.parseInt(input);
        return number;
    }

    public static void main(String[] args) {

        int n = readingInt();
        System.out.println(n);

        System.out.println("Done!");
    }
}
```

The main method calls the `readingInt()` method that supposedly reads an integer typed in to the keyboard. The program runs successfully if we enter a whole number.

Question:

An exception will be thrown instead if we do not enter a whole number. Try entering "abc". What is the name of the exception?

Answer:

`NumberFormatException`

Question:

What is the error message?

Answer:

For input string: "abc"

Notice that the error does not happen during compilation – the program compiles and runs successfully. The error happens during runtime, caused by user's unexpected behavior. We learn what kind of runtime error by reading the exception name and the error message.

Next, we will try simulate this in our method, `readingInt()`.

Consider the following program:

```
import java.util.Scanner;

public class ExceptionalLab {

    public static int readingInt() {

        Scanner sc = new Scanner(System.in);
        String input = sc.nextLine();

        if (!containAllDigit(input)) {
            String errorMessage = "For input string: \"" + input + "\"";
            throw new NumberFormatException(errorMessage);
        }

        int number = Integer.parseInt(input);
        return number;
    }

    private static boolean containAllDigit(String input) {
        for (int i = 0; i < input.length(); i++) {
            if (!Character.isDigit(input.charAt(i))) {
                return false;
            }
        }
        return true;
    }

    public static void main(String[] args) {
        int n = readingInt();
        System.out.println(n);

        System.out.println("Done!");
    }
}
```

The helper method `containAllDigit(String input)` will return true if and only if all the characters in the input strings are numbers.

We want to simulate the process called throwing an exception.

We throw an exception whenever something unexpected or abnormal event happens.

To simulate it, we first need to detect such event and we use the method `containAllDigit` to detect the event that not all of the characters in the input string are numbers.

Next, when that happens, we throw an exception.

Throwing an exception acts like returning a value – the execution of the method finishes after throwing an exception, the statements after the throwing statement are not executed.

It does not matter what the return value of the method is, we can use throw keyword to throw an exception.

Therefore, we need to check and detect the abnormal events first, with one or more if statements that will throw an exception.

We put the statements for the normal execution after that, that will continue what the method computes and returns as it should do accordingly – in our case, parsing the String into a number and return the number.

We need to choose the correctly named exception, because this is one way we convey the type of error. We choose `NumberFormatException`, because there is something wrong related formatting a number.

Question:

What is it exactly that we throw in the method?

Answer:

Notice that we throw an exception object, thus we need to use the new operator to create such exception object.

We can also pass a string into the exception constructor to further explain the error. We create a similar error message to the one we have seen before, and pass the string into the `NumberFormatException` constructor.

Question:

Now run the program, and give it a non-number string such as "abc".
What do you get? Does the program run and finish successfully?

Answer:

We get the same exception and its error message as before.

```
Exception in thread "main" java.lang.NumberFormatException: For input string: "abc"  
    at ExceptionalLab.readingInt (ExceptionalLab.java:13)  
    at ExceptionalLab.main (ExceptionalLab.java:31)
```

The program does not run and finish successfully – evidenced by the last statement of printing "Done!" is not executed. It ends abruptly – showing "BUILD FAILED" message in red in NetBeans.

How do we make sure that even though some unexpected/abnormal events happened, the program can still run and finish successfully?

We need to handle the exception – we show how to do that in the next section: how to handle the exception by catching it.

Lab Task 9.3 Exception Handling / Exception Catching

We modify the main method that calls the `readingInt()` method as follows:

```
public static void main(String[] args) {  
  
    try {  
        int n = readingInt();  
        System.out.println(n);  
    } catch (NumberFormatException nfe) {  
        System.out.println("You enter a non-number string!");  
        System.out.println(nfe.getMessage());  
    }  
  
    System.out.println("Done!");  
  
}
```

Question:

Now run the program, and give it a non-number string such as "abc".
What do you get? Does the program run and finish successfully?

Answer:

This time the program runs and finishes successfully, "Done!" is printed and it shows "BUILD SUCCESSFUL" despite being given an abnormal string that contains a non-number character.

Here are how we handle the exception by catching it using try-catch statement.

1. We put the risky method call that potentially throw an exception inside the try block – the call to the `readingInt()` method.
Not only that, we also put any statements that depend on the successful normal execution of that risky method call inside the try block – in our example above, the printing of `n`, it depends on the `readingInt()` successfully reading an integer string input.
2. If the risky method call throws an exception, instead of returning the correct value, the program execution will jump to the statements inside the catch block (the remaining statements below it in the try block will not be executed).
Thus, we put statements that needs to be executed should the unexpected happens, such as statements that can report and print messages related to the error.
The catch block catch the exception object thrown by specifying the type of the exception class, and give it a name – `nfe` in our case.
We can then access the message passed into the exception's constructor, by calling the method `getMessage()` on the caught exception object.

This is the end of CPT111 2223 Lab 9 Task Sheet.